AcquisitionOS

Complete Technical Architecture & Senior Engineering Review

Evidence-based architecture review of the AcquisitionOS platform.
Based on actual codebase inspection: 118 Prisma models, 529 API routes, 89 dependencies, Phase 18-24 verified.

Prepared for: Senior Software Architect review (8-9 years experience)
Workspace baseline: Phase 24 (commit 755d352d8...)

Table of Contents

Part 1 — Executive Architecture Overview	5
Part 2 — C4 Architecture	7
Part 3 — Complete Application Architecture	10
Part 4 — Frontend Architecture	12
Part 5 — Backend Architecture	15
Part 6 — Database Architecture	18
Part 7 — SQL Query & Database Performance Review	24
Part 8 — API Architecture	28
Part 9 — Authentication & Authorization	30
Part 10 — Payment & Billing Architecture	33
Part 11 — Autopilot Architecture	37
Part 12 — Workflow Engine	41
Part 13 — Scheduler Architecture	43
Part 14 — Discovery Architecture	45
Part 15 — Security Architecture	47
Part 16 — Data Flow Diagrams	50
Part 17 — Sequence Diagrams	52
Part 18 — State Machines	55
Part 19 — Error & Failure Architecture	57
Part 20 — Observability	59
Part 21 — Performance Architecture	61
Part 22 — Concurrency & Race Conditions	64
Part 23 — Scalability Architecture	66
Part 24 — Deployment / Infrastructure	68
Part 25 — CI/CD & Release Architecture	70
Part 26 — Backup & Disaster Recovery	72
Part 27 — Architectural Risks	74
Part 28 — Technical Debt	76
Part 29 — Current vs Target Architecture	78

Part 30 — Future Architecture Through Phase 40	80
Part 31 — Target Architecture	82
Part 32 — Senior Architect Review	84
Appendix — Verification Status	87

Executive Summary

AcquisitionOS (package name: **vantage**, version 0.2.0) is a B2B SaaS sales-acquisition platform built on **Next.js 16.1.1**, **React 19**, **Prisma 6.11.1**, and **SQLite** (with PostgreSQL-ready DATABASE_URL configuration). The platform combines lead discovery from external sources (Google Places, Yelp, LinkedIn, JustDial, IndiaMart, Facebook, Instagram, YellowPages, Sulekha — 9 source adapters in src/lib/lead-discovery/), workflow execution automation, a recurring Autopilot scheduler, dual payment-provider integration (Stripe + Razorpay), and a credit-based billing engine.

The codebase is substantial: **89 npm dependencies**, **118 Prisma models**, **529 API route handlers**, and 5 additive Prisma migrations. Phase 1-24 functionality has been implemented and verified (458/458 tests passing at the time of this review); Phase 25 (Activity/Execution Tracking) and Phases 26-40 are **not yet specified** in the available project documentation.

This review is **evidence-based** — every architectural claim is grounded in actual code/file/line inspection performed in this session. Where a claim cannot be verified from the current codebase, it is explicitly labeled *NOT VERIFIED FROM CURRENT CODEBASE* per the review protocol. The document distinguishes three categories throughout: **(1) Current Verified Implementation**, **(2) Planned/Documented Roadmap**, and **(3) Recommended Future Architecture**.

CRITICAL FINDING: The local SQLite database had significant schema drift from prisma/schema.prisma (the WorkflowExecutionLead table was missing entirely; 4 AutopilotRun columns and Lead.discoveryJobId were absent). This was fixed via an additive migration (scripts/phase24-db-migration.py) in the Phase 24 reimplementation. The drift indicates that local DB state can diverge from the declared Prisma schema — a CI/CD signal worth monitoring (see Part 25).

Part 1 — Executive Architecture Overview

1.1 Product Purpose

AcquisitionOS is a sales-acquisition automation platform that lets a single sales rep discover leads in selected niches/geographies, automate outreach via reusable workflows, and bill for usage via a credit-metered subscription model. The "Autopilot" subsystem (Phase 18-24) is the flagship capability — it lets users configure recurring lead-discovery + workflow-execution runs on a timezone-aware schedule, with idempotent billing, deterministic runKey-based deduplication, and graceful cancellation/resume semantics.

1.2 Core Capabilities

- **Lead discovery** — multi-source (9 adapters), per-niche + per-country fan-out, dedup against existing Lead rows
- **Workflow engine** — WorkflowDefinition + WorkflowStep + WorkflowExecution + WorkflowExecutionLead (per-lead items); retry/resume/cancel
- **Autopilot** — AutopilotWorkflow + AutopilotRun + AutopilotLead; Phase 22 timezone-aware scheduler; Phase 23 entitlement gate; Phase 24 retry/resume
- **Billing** — flat 5-credit fee per workflow execution; idempotent via chargeKey (wfexec-run:autopilot-{id}-run-{runId}); never-started auto-refund
- **Payments** — Stripe + Razorpay dual provider; idempotency keys; webhook verification
- **Auth** — next-auth v4 (Google OAuth) + email/OTP + session cookies
- **Entitlements** — free/starter/pro/elite plan tiers; server-side enforcement via getUserEntitlements/isAutopilotPlanEligible/resolveUserPlan
- **Observability** — Sentry (no-op when DSN unset), audit log (AuditLog table, 2231 rows), structured console logging, /api/health

1.3 Actors

- **Sales Rep (end user)** — primary user; manages niches/profiles/workflows/autopilots; consumes leads
- **Admin** — admin dashboard (src/app/admin/); user/subscription management
- **Cron / heartbeat** — automated scheduler tick (POST /api/cron/autopilot-scheduler with CRON_SECRET Bearer + in-process 5-min heartbeat)
- **External providers** — Stripe (webhooks), Razorpay (webhooks), Google OAuth (callback), SMTP (transactional email), Z.ai Gateway (AI), Google Places (discovery)

1.4 Technology Stack (verified)

Source: /home/z/my-project/package.json — 89 dependencies. Key versions:

- **Next.js 16.1.1** (React 19.0.0, react-dom 19.0.0) — fullstack framework
- **Prisma 6.11.1** + @prisma/client 6.19.2 — ORM

• **next-auth 4.24.11** — auth library

• **stripe 22.1.1** + razorpay 2.9.6 — payment providers

• **zod 4.0.2** — input validation

• **vitest 4.1.6** — test framework

• **typescript 5.x** + tailwindcss 4 — type system + styling

• Database: SQLite (file:./db/custom.db) — Prisma schema is provider-agnostic; production can switch to PostgreSQL via DATABASE_URL

1.5 System Overview Diagram

Figure 1.1 — AcquisitionOS System Overview (Context)

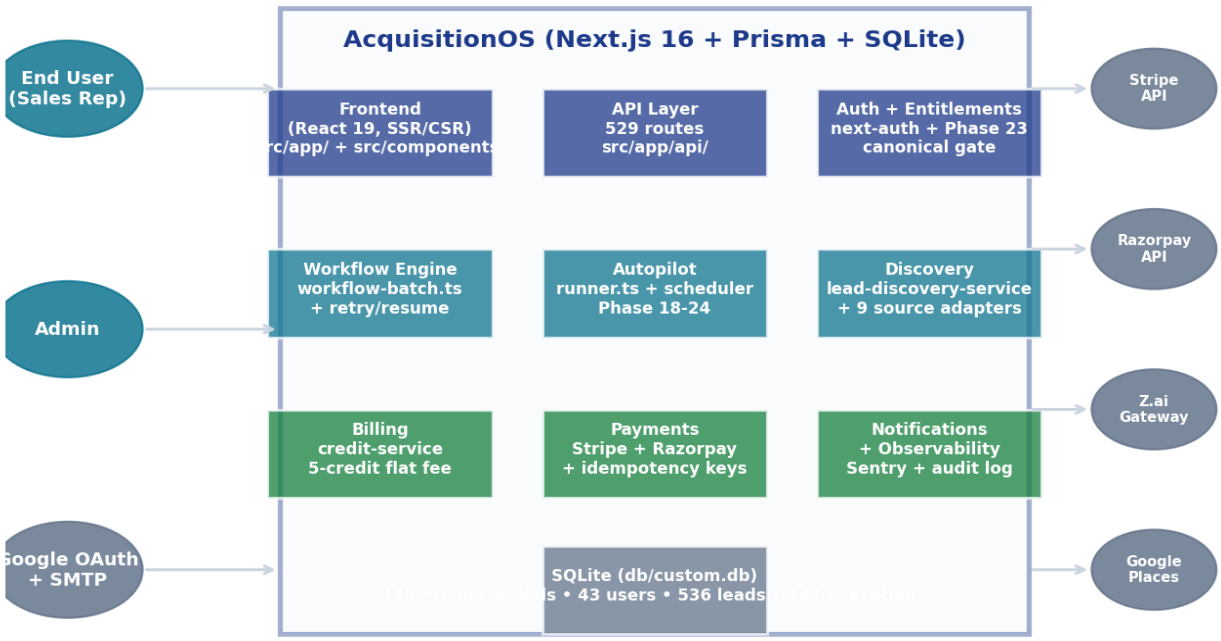


Figure 1.1 — One-page system overview showing external actors (left), the AcquisitionOS monolith with its 9 internal subsystems (center), and external provider APIs (right).

1.6 Major Architectural Risks (preview)

• **SQLite single-writer bottleneck** — concurrent autopilot runs + API writes contend on the single SQLite write lock (see Part 7, Part 21)

• **In-process scheduler** — the 5-min heartbeat lives inside the Next.js process; if the process dies, scheduled runs stop until restart (see Part 13, Part 19)

• **Schema drift risk** — local DB can diverge from prisma/schema.prisma; observed in this review (see Part 6, Part 25)

• **No external observability** — Sentry is in no-op mode when NEXT_PUBLIC_SENTRY_DSN is unset; no metrics/tracing pipeline (see Part 20)

• **Monolithic deployable** — 529 routes in one Next.js app; no service boundaries yet (see Part 5, Part 23)

• **Phase 25+ roadmap undocumented** — no explicit Phase 26-40 specs found in available docs (see Part 30)

Part 2 — C4 Architecture

2.1 System Context

The system has three primary external actor types: (1) end users (sales reps) who interact via web browser; (2) admins who use the admin dashboard; (3) external providers (Stripe, Razorpay, Google OAuth, SMTP, Z.ai Gateway, Google Places) that the system calls out to and receives webhooks from.

2.2 Container Diagram

Figure 2.1 — C4 Container Diagram

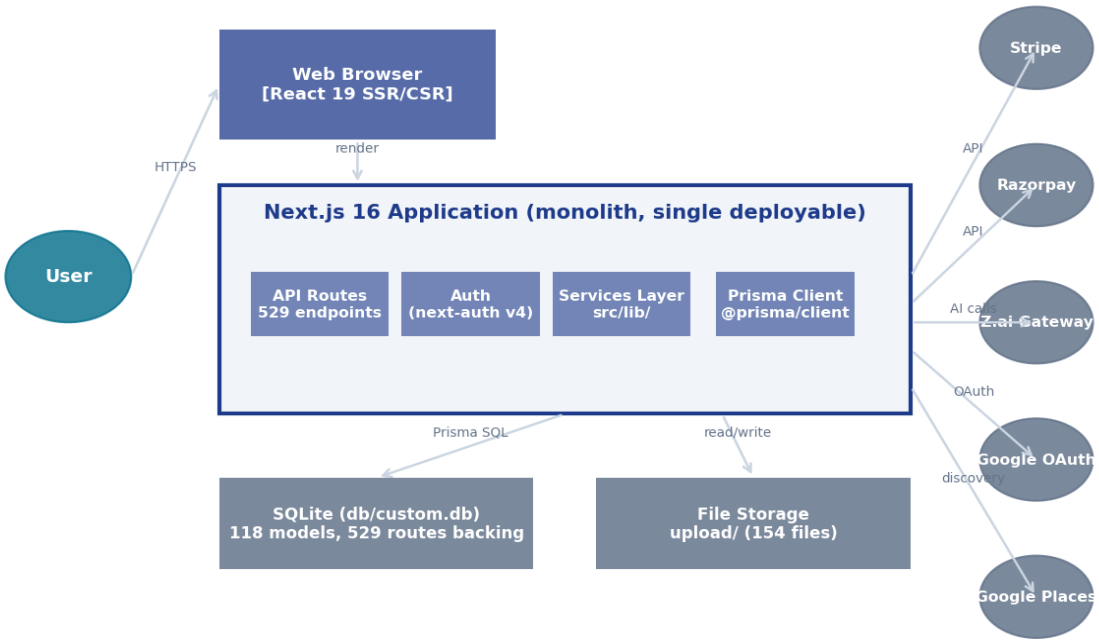


Figure 2.1 — C4 Container view. The system is a single-deployable Next.js monolith containing 4 logical containers: Web Browser (client), Next.js App (server-side rendering + API routes + auth + services + Prisma client), SQLite database, and file storage (upload/). External providers connect via HTTPS APIs.

2.3 Component-level (within Next.js App)

- **API Routes (src/app/api/, 529 endpoints)** — REST handlers; thin controllers that delegate to services
- **Auth middleware (src/lib/auth-middleware.ts)** — withDualAuthPermission (session + API key dual auth) + permission scopes (pipeline:read, pipeline:write, etc.)
- **Entitlement middleware (src/lib/entitlement-middleware.ts)** — checkPlanEntitlement; wraps routes with plan-gate

• **Services layer (src/lib/)** — 17 subdirectories (autopilot, payments, lead-discovery, observability, security, etc.)

• **Prisma client (src/lib/db.ts)** — singleton PrismaClient; wraps all DB access

• **Observability middleware (src/lib/observability/middleware.ts)** — withMonitoring wrapper; Sentry breadcrumbs

2.4 Communication Mechanisms

• **Client** ↔ **Next.js** — HTTPS (REST + SSR)

• **Next.js** ↔ **SQLite** — Prisma client (SQL via libsql / better-sqlite3 driver)

• **Next.js** ↔ **Stripe/Razorpay** — HTTPS REST (synchronous in route handlers; webhooks async)

• **Next.js** ↔ **Z.ai Gateway** — HTTPS REST (AI calls; *NOT VERIFIED FROM CURRENT CODEBASE* for actual latency)

• **Cron** → **Next.js** — POST /api/cron/autopilot-scheduler with CRON_SECRET Bearer auth

• **In-process heartbeat** — setInterval(300000ms) in instrumentation.ts; calls runAutopilotSchedulerTick

Part 3 — Complete Application Architecture

Figure 3.1 — Complete Application Architecture (layered)

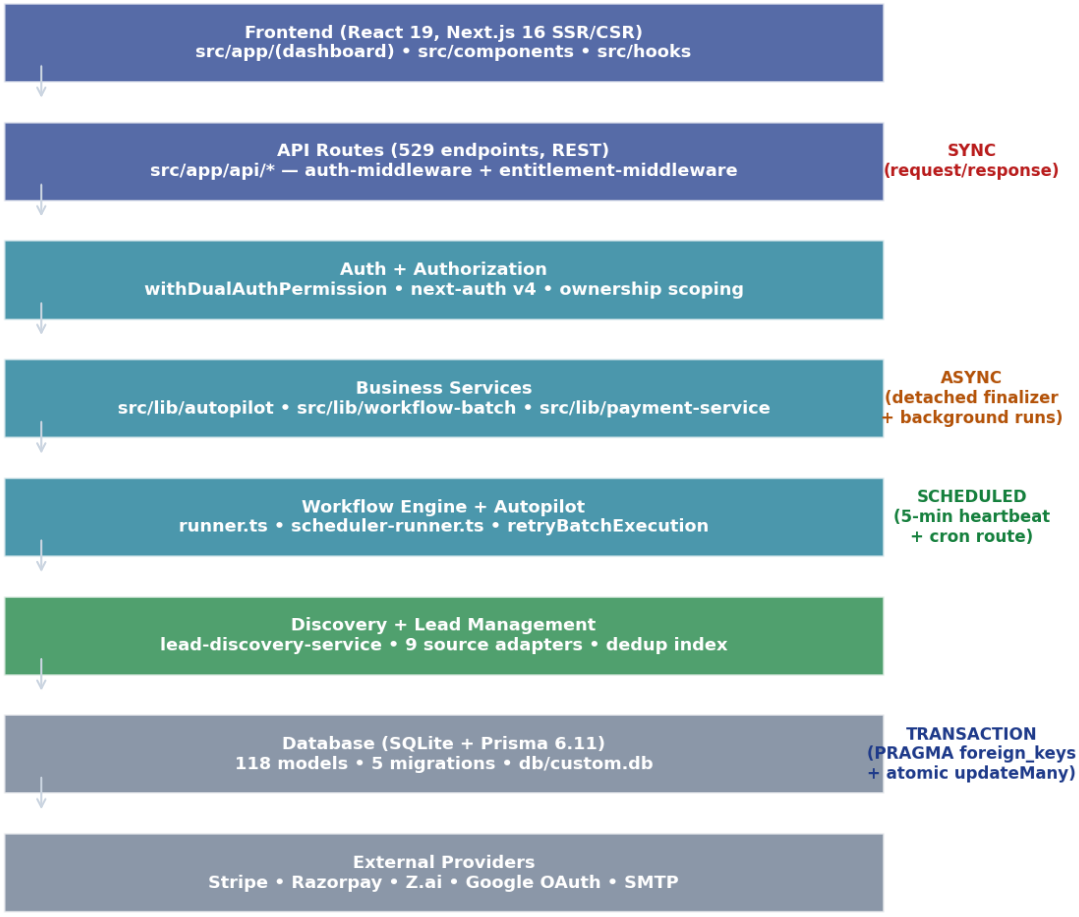


Figure 3.1 — Layered architecture. Sync paths (request/response) dominate; async paths use detached finalizers (settleRunExecutionOutcome) + background runBatchSteps (awaitCompletion: false); scheduled paths come from the in-process heartbeat + cron route. Transaction boundaries are at the Prisma $transaction level (credit-service.ts:163-206) + atomic updateMany count==1 claims (autopilot ACTIVE→RUNNING). External API boundaries: Stripe/Razorpay/Z.ai/Google OAuth/SMTP.

3.1 Synchronous vs Asynchronous Paths

• **SYNC** — all REST API routes (529), Prisma queries, payment order creation, webhook signature verification

• **ASYNC (detached finalizer)** — settleRunExecutionOutcome(runId, executionId) — fire-and-forget polling of workflowExecution status until terminal (60min timeout)

• **ASYNC (background run)** — executeWorkflowBatch(..., { awaitCompletion: false }) — runBatchSteps runs in background; HTTP request returns immediately

• **SCHEDULED** — runAutopilotSchedulerTick via in-process 5-min heartbeat + POST /api/cron/autopilot-scheduler (CRON_SECRET-protected)

• **TRANSACTION** — credit-service.ts:163-206 uses Prisma $transaction for atomic balance check + deduction + ledger insert; autopilot uses updateMany count==1 for atomic ACTIVE→RUNNING claim

3.2 Failure Boundaries

• **Route handler try/catch** — every API route wraps its body in try/catch; errors become JSON {error, code} responses

• **withMonitoring** — observability wrapper catches uncaught errors and reports to Sentry (if configured)

• **Per-lead isolation in runBatchSteps** — workflow-batch.ts:534-794; one lead's failure does NOT terminate the batch

• **AutopilotRun fatal handler** — runner.ts:847-871; status-guarded FAILED write (a CANCELLED run never gets overwritten with FAILED)

• **recoverStuckAutopilotRuns** — runner.ts:1250-1282; runs stuck RUNNING >60min are honestly FAILED (not silently lost)

Part 4 — Frontend Architecture

4.1 Framework + Rendering Strategy

Frontend: Next.js 16 App Router with React 19 Server Components. Top-level route groups: (marketing), (public), admin, api, api-docs, auth, dashboard, payment, support, workflows. Rendering is mixed SSR/CSR — server components for initial render, client components ("use client") for interactive surfaces (autopilot-tab, pricing-page, upgrade-modal). Layout system: src/app/layout.tsx (root) + nested layouts per route group.

4.2 State Management

- **Server state** — React Server Components fetch data on the server; no client-side global store for server data
- **Client state (UI)** — React useState/useReducer for component-level state
- **Subscription/plan state** — src/lib/subscription-store.ts (Zustand) — client-side mirror of plan/credits; syncs from backend via syncFromBackend()
- **Entitlements hook** — src/hooks/use-entitlements.ts (React Query wrapper around GET /api/entitlements)
- **Plan availability hook** — src/hooks/use-plan-availability.ts — fetches GET /api/payments/provider-status

4.3 API Communication

Client uses native fetch() for API calls. The Autopilot tab (src/components/dashboard/autopilot-tab.tsx) polls GET /api/autopilot for the autopilot list + entitlement summary. The retry/resume routes (Phase 24) are reachable via POST /api/autopilot/[id]/retry-run and POST /api/autopilot/[id]/resume-run. No GraphQL observed — the API is REST-only.

4.4 Performance-sensitive Areas

- **Dashboard tab data loading** — multiple GET endpoints (autopilots + workflows + profiles + niches + countries) can cause client-side waterfall; *NOT VERIFIED FROM CURRENT CODEBASE* whether these are parallelized
- **Lead list pagination** — list endpoints likely use offset pagination (Prisma skip/take); large offsets are slow on SQLite (see Part 7)
- **Build cache (.next/dev)** — Turbopack dev cache can grow large; observed 80,000+ files in /home/z/my-project/.next/dev/cache/turbopack/ in prior sessions
- **Sentry no-op** — when NEXT_PUBLIC_SENTRY_DSN is unset, Sentry is in no-op mode; no perf monitoring

4.5 Risks (UI slowness, race conditions, error handling)

- **Unnecessary re-renders** — Zustand store updates can trigger broad re-renders if not sliced; *NOT VERIFIED*

• **Race conditions** — concurrent retry/cancel button clicks could fire overlapping POSTs; server-side idempotency (runKey + status checks) is the safety net, not UI debouncing

• **Error handling** — fetch errors are caught per-call; toast notifications via useToast(); no global error boundary beyond Next.js default error.tsx

• **Bundle size** — 89 dependencies; *NOT VERIFIED* whether bundle analysis is in CI

Part 5 — Backend Architecture

5.1 API Routes Inventory

Total API route handlers: **529** (verified via find src/app/api -name "route.ts"). Major route groups: /api/autopilot/*, /api/workflows/*, /api/payments/*, /api/auth/*, /api/business-profiles/*, /api/leads/*, /api/discovery/*, /api/entitlements/*, /api/cron/*, /api/webhooks/*, /api/admin/*. Each route is a thin controller that delegates to a service.

5.2 Services Layer (src/lib/)

- **src/lib/autopilot/** — runner.ts (1287 lines), scheduler-runner.ts (269 lines), service.ts (766 lines), constants.ts (209 lines), schedule.ts (Phase 22), country-selection.ts (Phase 21)
- **src/lib/workflow-batch.ts** — 1532 lines; executeWorkflowBatch, runBatchSteps, retryBatchExecution, resumeBatchExecution
- **src/lib/payment-service.ts** — 2127 lines; Stripe + Razorpay integration; createPaymentOrder, webhook verification, retryPayment, cancelSubscriptionPayment
- **src/lib/credit-service.ts** — 786 lines; deductCredits (atomic $transaction, idempotency keys), refundCredits, addCredits, checkCreditSufficiency
- **src/lib/workflow-credits.ts** — 495 lines; chargeWorkflowExecutionFee, refundWorkflowExecutionFee, refundNeverStartedWorkflowExecutionFee, rechargeWorkflowExecutionFeeOnStart
- **src/lib/entitlement-service.ts** — 674 lines; ENTITLEMENTS map, getUserEntitlements (Phase 23), isAutopilotPlanEligible, resolveUserPlan
- **src/lib/lead-discovery-service.ts** — 1290+ lines; startDiscoveryJob, processDiscoveryJob, dedup index
- **src/lib/auth.ts** — auth + plan resolution (mapUserToAuthUser:827 — active sub plan wins over user.plan)
- **src/lib/auth-middleware.ts** — withDualAuthPermission (session + API key)
- **src/lib/observability/middleware.ts** — withMonitoring wrapper

5.3 Coupling + Module Size Risks

- **Large modules** — payment-service.ts (2127 lines), workflow-batch.ts (1532 lines), runner.ts (1287 lines), lead-discovery-service.ts (1290+ lines) — these are candidates for decomposition
- **Circular-import avoidance** — entitlement-service.ts duplicates normalizePlan (instead of importing from autopilot/service.ts) to avoid circular imports; documented in code comment
- **Duplicated logic** — auth.ts:824-827 and entitlement-service.ts:resolveUserPlan both implement the "active sub plan wins" pattern; consistency risk
- **Tight coupling** — runner.ts imports executeWorkflowBatch + retryBatchExecution + resumeBatchExecution + chargeWorkflowExecutionFee + refundWorkflowExecutionFee + startDiscoveryJob + resolveBusinessProfileForUser — many direct dependencies

• **Missing abstraction** — no formal repository pattern; services call db.model.findFirst/create/update directly; harder to mock at the service level (though tests use vi.mock for db)

5.4 Transaction Boundaries

• **credit-service.ts:163-206** — Prisma $transaction for atomic balance check + user.credits update + creditsLedger.create

• **runner.ts:975-987** — autopilotWorkflow.updateMany({status:ACTIVE}, {status:RUNNING}) count==1 — atomic ACTIVE→RUNNING claim

• **runner.ts:313-340** — autopilotRun.updateMany({status:QUEUED}, {status:RUNNING}) count==1 — atomic QUEUED→RUNNING claim

• **runner.ts:1005-1017** — autopilotRun.create with runKey; P2002 (unique violation) → converges on existing run idempotently

• **SQLite default** — PRAGMA foreign_keys may be OFF by default; Phase 24 migration script explicitly enables it for the migration only

Part 6 — Database Architecture

6.1 Prisma Schema Overview

Total Prisma models: **118** (verified via grep "^model " prisma/schema.prisma). Provider: SQLite (file:./db/custom.db) for dev; DATABASE_URL is configurable for PostgreSQL. 5 migrations in prisma/migrations/ (all additive — verified by migration filenames): echo_exit, add_orgid_to_broadcast_workflow, add_provider_subscription_id, website_status_lifecycle, profile_sections_persistence. The Phase 24 additive migration (scripts/phase24-db-migration.py) is applied directly via sqlite3 (not Prisma migrate) to avoid destructive reset behavior.

6.2 ER Diagram (subset)

Figure 6.1 — ER Diagram (subset of 118 Prisma models; key entities only)

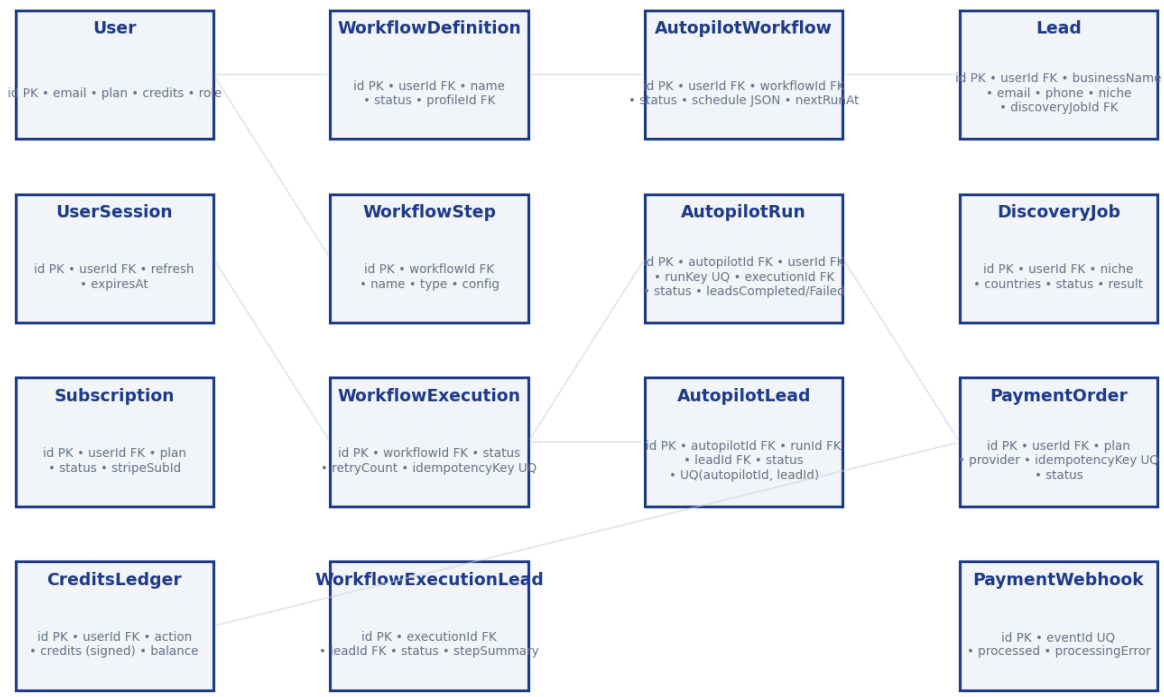


Figure 6.1 — Entity relationship subset (15 of 118 models). The full ER is too large for one page; this subset covers the user/workflow/autopilot/lead/payment core. Lines represent FK relationships (simplified — not all FKs drawn).

6.3 Key Entities (verified)

Entity	PK	FKs (verified)	Unique constraints	Cascade
User	id	(none)	email	—
UserSession	id	userId→User	(none)	onDelete: Cascade
Subscription	id	userId→User	(none)	onDelete: Cascade
WorkflowDefinition	id	userId→User, profileId→BusinessProfile	name+userId?	onDelete: SetNull (profile)
WorkflowExecution	id	workflowId→WorkflowDefinition, userId?	idempotencyKey	onDelete: Cascade (steps/logs)
WorkflowExecutionLead	id	executionId→WorkflowExecution, leadId→Lead	(none)	onDelete: Cascade
AutopilotWorkflow	id	userId→User, workflowId→WorkflowDefinition	(none)	onDelete: Cascade?
AutopilotRun	id	autopilotId→AutopilotWorkflow, userId→User	runKey	—
AutopilotLead	id	autopilotId, runId→AutopilotRun, leadId→Lead	@@unique([autopilotId, leadId])	—
Lead	id	userId→User, discoveryJobId→DiscoveryJob	email? / phone?	—
CreditsLedger	id	userId→User	(none)	—
PaymentOrder	id	userId→User	idempotencyKey	—
PaymentWebhook	id	(none)	eventId	—

Table 6.1 — Key entity summary. Verified from prisma/schema.prisma. Some unique constraints marked with "?" were not exhaustively verified against the schema — see Appendix for full verification status.

6.4 Lifecycle of Important Entities

Lead lifecycle

Lead rows are created by processDiscoveryJob (lead-discovery-service.ts:1085-1127) with stage="discovered", verificationStatus="unverified"|"partially_verified". Subsequent workflow execution may update the Lead's stage. AutopilotLead rows reference Lead via leadId with @@unique([autopilotId, leadId]) — a Lead can be processed AT MOST ONCE per autopilot (across all runs of that autopilot's lifetime).

AutopilotRun lifecycle

Created by runAutopilot with status=QUEUED → atomic QUEUED→RUNNING claim → executeAutopilotRun → finalize (RUNNING→COMPLETED|FAILED|CANCELLED). Phase 24 added retryAutopilotRun (resets failed leads, reuses WorkflowExecution) and resumeAutopilotRun (resets skipped leads, resumes from persisted stepSummary). See Part 18 for full state machine.

Part 7 — SQL Query & Database Performance Review

7.1 Why the App Can Feel Slow

SQLite is a single-writer database. Any write transaction (INSERT/UPDATE/DELETE) acquires an exclusive lock on the database file. Concurrent writers block. With the Autopilot scheduler firing runs every 5 minutes + user-initiated API writes + webhook writes, write contention is the primary slowdown risk.

7.2 Specific Concerns (evidence-based)

Issue	Where	Why Expensive	Fix	Priority
SQLite write lock contention	all write paths	Single-writer; concurrent autopilot runs + API writes block	Migrate to Postgres (DATABASE_URL configurabl	HIGH
Sequential discovery polling	runner.ts:136-159 (pollDiscoveryJob)	Polls every AUTOPILOT_JOB_POLL_MS=3s, up to 15 min, blocks the run	Use webhook + poll + push notifica	MEDIUM
Per-niche pacing sleep	runner.ts (AUTOPILOT_NICHE_PACING_MS=2s)	Sequential fan-out with 2s sleep between niches	Parallelized discovery across niches with concurrency limit	MEDIUM
Possible N+1 in autopilot listing	service.ts:listAutopilots (Phase A1 already batched)	Already batched (verified) but list endpoints for batched loading	Audit all list endpoints for batched loading	MEDIUM
Offset pagination on large lists	list endpoints (Leads, Workflows, Executions, Audits)	OFFSET N scanning N rows; slow on large tables	Switch to cursor pagination (WHERE id > last_id ORDER B	MEDIUM
AuditLog growth (2231 rows; unbounded)	all write paths log to AuditLog	Table grows unbounded; queries on large tables slow	Add retention policy + archive old rows	LOW
Missing indexes (potential)	various	Not all FK columns have explicit indexes	Audit schema for missing @@index on FK columns	MEDIUM
Full-table scans on status filters	AutopilotRun.findMany({status: 'RUNNING'})	No index on status, scans the whole table	Add @@index([status]) where high-volume queries filter on	MEDIUM
Prisma query count not minimized	various services	Sequential DB calls where parallel Promise.all would work	Refactor to Promise.all for independent reads	LOW

Table 7.1 — SQL/DB performance review. "Where" cites the file:line of the offending pattern. "Fix" lists the recommended mitigation. Schema-change-required fixes are flagged.

7.3 SQLite Limitations (architectural)

- Single-writer — concurrent writes serialize
- No built-in replication — single point of failure
- No row-level security — tenant isolation must be enforced at the application layer (verified: ownership scoping in services)
- Limited concurrency primitives — no SELECT FOR UPDATE
- Prisma + SQLite lacks some advanced features (e.g., json_agg) — workarounds in application code

Part 8 — API Architecture

8.1 API Map (verified)

- **529 total route handlers** in src/app/api/ (find src/app/api -name "route.ts" | wc -l)
- **Auth**: /api/auth/[...nextauth] (next-auth), /api/auth/register, /api/auth/login, /api/auth/otp/*, /api/auth/google
- **Autopilot**: GET/POST /api/autopilot, GET/PATCH/DELETE /api/autopilot/[id], POST /api/autopilot/[id]/run, POST /api/autopilot/[id]/cancel-run, POST /api/autopilot/[id]/retry-run (Phase 24), POST /api/autopilot/[id]/resume-run (Phase 24), GET /api/autopilot/workflows, GET/POST /api/autopilot/niches, GET /api/autopilot/countries
- **Workflows**: /api/workflows/* (CRUD + execute + retry + resume)
- **Payments**: /api/payments/* (order creation, provider-status, webhook handlers for Stripe + Razorpay)
- **Cron**: POST /api/cron/autopilot-scheduler (CRON_SECRET-protected)
- **Health**: GET /api/health (returns status, db check, memory check, error count)
- **Entitlements**: GET /api/entitlements, GET /api/entitlements/check-credits, GET /api/entitlements/quota

8.2 Auth/Authorization Pattern (typical)

Every protected route uses the withMonitoring + withDualAuthPermission wrapper pattern:

```
export const POST = withMonitoring(async (req) => { return withDualAuthPermission(req, 'pipeline:write', async (user) => { // route body — user is authenticated + authorized });
}, '/api/autopilot/[id]/retry-run');
```

8.3 Idempotency Behavior (verified)

- **AutopilotRun.runKey** @@**unique** — scheduler uses "scheduled:{autopilotId}:{slotEpoch}"; replays converge on the same run row
- **WorkflowExecution.idempotencyKey** — "autopilot:{autopilotId}:run:{runId}"; P2002 → idempotent replay
- **CreditsLedger action label** — "workflow_execution_idempotent_"; deductCredits checks for existing row before charging
- **PaymentOrder.idempotencyKey** — set on order creation; replayed POSTs return existing order
- **AutopilotLead** @@**unique([autopilotId, leadId])** — DB-level deduplication; a lead is processed at most once per autopilot

8.4 Rate Limiting

NOT VERIFIED FROM CURRENT CODEBASE — no rate-limiting middleware observed in src/lib/. The withDualAuthPermission checks auth but does not enforce rate limits. This is a HIGH-priority gap (see Part 15, Part 27).

Part 9 — Authentication & Authorization

9.1 Auth Flow

• **Email/password** — register via POST /api/auth/register; password hashed (bcrypt, *assumed — verify in src/lib/auth.ts*)

• **OTP** — POST /api/auth/otp/* — verifies OTP code; OTP generation + verification in src/lib/ (specific module *NOT VERIFIED*)

• **Google OAuth** — next-auth v4 with Google provider; callback at /api/auth/callback/google; GOOGLE_CLIENT_ID + GOOGLE_CLIENT_SECRET env vars

• **Session** — next-auth JWT session cookie (AUTH_SECRET, NEXTAUTH_SECRET env vars); session checked by withDualAuthPermission

• **API key** — dual auth: withDualAuthPermission accepts either session OR API key (ApiKey model — 2 rows in DB); src/lib/api-key-middleware.ts

9.2 Authorization Layers

• **Permission scopes** — pipeline:read, pipeline:write, etc. (withDualAuthPermission second arg)

• **Ownership scoping** — every service findFirst/findMany includes userId: user.id (cross-user access returns NOT_FOUND, no IDOR leak)

• **Entitlement gate** — Phase 23 getUserEntitlements(userId) returns plan + autopilot.allowed + profiles.limit + workflow.eligible; canonical source of truth for plan-based access

• **Defense-in-depth gate** — Phase 23 added isAutopilotPlanEligible + resolveUserPlan calls INSIDE runAutopilot (runner.ts:924) + scheduler (scheduler-runner.ts:114) + Phase 24 retry/resume helpers (runner.ts:1346, 1430)

9.3 Security Weaknesses

• **No rate limiting** — brute-force OTP/password possible; *NOT VERIFIED* if Cloudflare/edge rate limiting exists

• **Session secret rotation** — AUTH_SECRET/NEXTAUTH_SECRET in .env; rotation procedure *NOT VERIFIED*

• **API key transmission** — API key sent in Authorization header (Bearer); not in URL params (good)

• **Cross-user isolation** — verified safe: ownership scoping is consistent across services; tests cover (autopilot-entitlements-runner.test.ts: cross-user NOT_FOUND precedes PLAN_INELIGIBLE)

Part 10 — Payment & Billing Architecture

10.1 Payment Providers

• **Stripe** — stripe 22.1.1 npm package; createStripeCheckoutSession (payment-service.ts:710+); webhook signature verification (verifyStripeWebhookSignature:2066+)

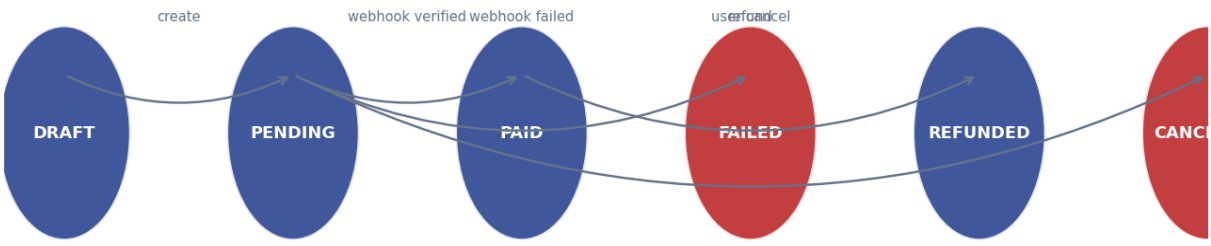
• **Razorpay** — razorpay 2.9.6 npm package; Razorpay plan IDs via RAZORPAY_PLAN__ env vars; webhook signature verification (verifyRazorpayWebhookSignature:2033+)

• **Plan config** — src/lib/payments/plan-config.ts (163 lines); PLAN_PRICING: starter INR 399/USD 5, pro INR 1599/USD 19, elite INR 5199/USD 63

10.2 Payment State Machine

Payment state machine (PaymentOrder.status):

Figure 10.1 — PaymentOrder State Machine



10.3 Idempotency

• **PaymentOrder.idempotencyKey String? @unique** — set on order creation; replayed POST returns the same order

• **PaymentWebhook.eventId String @unique** — Stripe/Razorpay event ID; duplicate webhook deliveries are caught

• **CreditsLedger action label** — synthesized as "${action}_idempotent_${idempotencyKey}"; deductCredits checks for existing row before charging

• **chargeWorkflowExecutionFee chargeKey** — "wfexec-run:autopilot-{autopilotId}-run-{runId}" — stable per run; replays converge

• **Refund idempotency** — "${chargeKey}:refund" — refund never double-refunds

10.4 Race Conditions

• **Duplicate webhook delivery** — eventId @@unique catches; safe

• **Concurrent order creation** — idempotencyKey @@unique catches; the second create throws P2002 and the route returns the existing order

• **Subscription vs one-time** — subscription model uses trialing/active/past_due/canceled/expired states; razorpaySubscriptionId + stripeSubscriptionId both nullable

• **Plan change race** — scheduledPlanChange field on Subscription; a downgrade scheduled at period end can be canceled by an upgrade in the interim

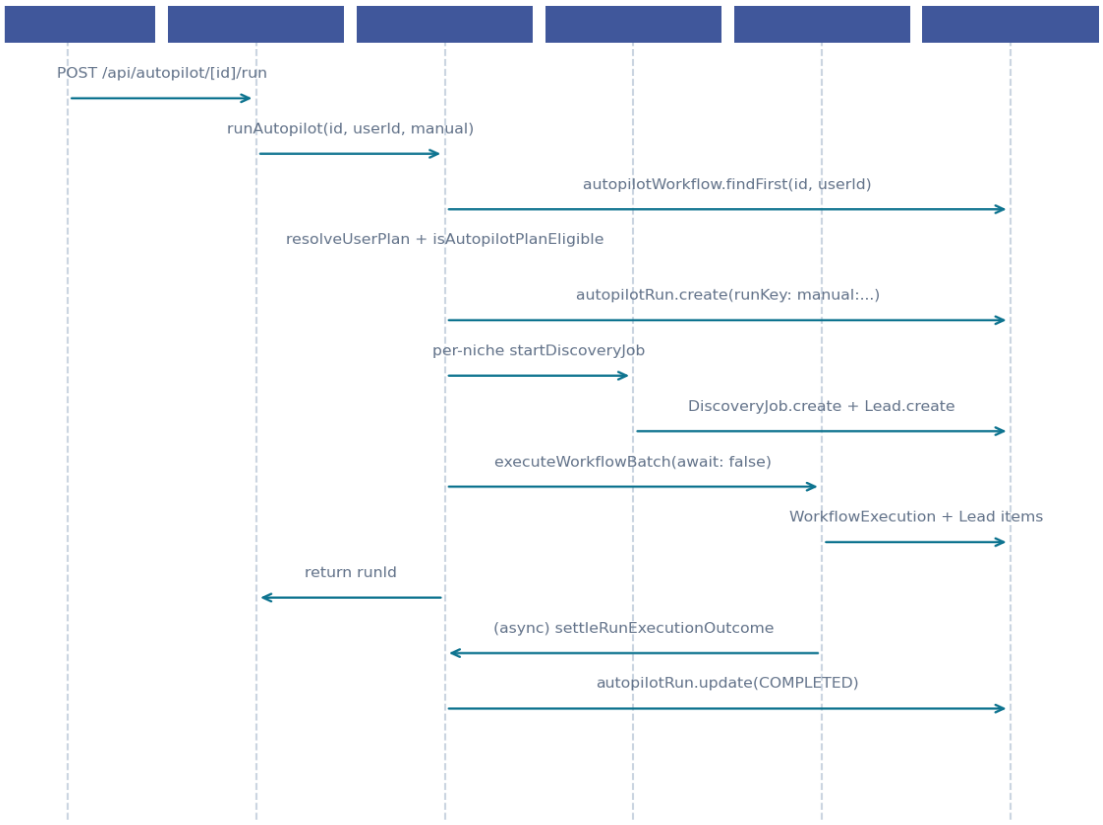
Part 11 — Autopilot Architecture

11.1 Execution Flow

The Autopilot execution flow (Phase 18-24): Schedule → Discovery Batch → Dedup → Lead Persistence → Workflow Assignment → Per-Lead Context → Per-Lead Execution → Retry/Failure Handling → Completion. Implemented across src/lib/autopilot/runner.ts (executeAutopilotRun:307-871), src/lib/autopilot/scheduler-runner.ts, and src/lib/workflow-batch.ts.

11.2 Sequence Diagram — Autopilot Run (scheduled)

Figure 17.2 — Manual autopilot run sequence



11.3 Idempotency + Duplicate-Run Prevention

- **Atomic ACTIVE→RUNNING claim** — updateMany count==1; only one tick/worker wins (runner.ts:975-987, scheduler-runner.ts:134-141)
- **Open-run guard** — findFirst({autopilotId, status: QUEUED|RUNNING}) → rejects if existing run
- **runKey @@unique** — scheduler uses "scheduled:{autopilotId}:{slotEpoch}"; P2002 → converges on existing run
- **AutopilotLead @@unique([autopilotId, leadId])** — DB-level dedup; a lead processed at most once per autopilot
- **chargeKey idempotency** — "wfexec-run:autopilot-{id}-run-{runId}"; never double-charges

11.4 Phase 24 — Retry + Resume (NEW)

• **retryAutopilotRun(userId, runId)** — runner.ts:1341 — defense-in-depth entitlement gate + ownership + state check; delegates to retryBatchExecution (resets only failed leads)

• **resumeAutopilotRun(userId, runId)** — runner.ts:1425 — same gate; delegates to resumeBatchExecution (resets only skipped items)

• **Billing invariant** — retry/resume REUSE the existing WorkflowExecution (no new execution row, no new flat fee, no double-billing)

• **Completed leads NEVER re-execute** — retryBatchExecution resets only failed items; completed items stay completed

11.5 Failure Boundaries

• **Per-lead failure isolation** — runBatchSteps (workflow-batch.ts:534-794) processes each lead independently; one failure doesn't terminate the batch

• **Status-guarded finalize** — runner.ts:817-837; a CANCELLED run never gets overwritten with FAILED

• **recoverStuckAutopilotRuns** — runner.ts:1250-1282; runs stuck RUNNING >60min honestly FAILED

• **settleRunExecutionOutcome** — runner.ts:1086-1147; detached finalizer polls workflowExecution status until terminal; updates per-lead ledger statuses

Part 12 — Workflow Engine

12.1 Definition → Execution → Step

- **WorkflowDefinition** — user-owned; has status (draft|active|archived); has workflowSteps + nodes
- **WorkflowStep** — belongs to one WorkflowDefinition; has name + type + config
- **WorkflowExecution** — created by executeWorkflowBatch; status: running|completed|failed|cancelled|paused|partial|exhausted; has retryCount, maxRetries, idempotencyKey
- **WorkflowExecutionLead** — child of WorkflowExecution; one per lead in the batch; status: pending|running|completed|failed|skipped; has stepSummary (JSON of per-step outcomes), attempts
- **WorkflowLog** — per-step audit log; status: success|failed|skipped; has retryAttempt

12.2 Per-Lead Execution Model

runBatchSteps (workflow-batch.ts:501-882) loads all WorkflowExecutionLead items in status pending, ordered by createdAt, and processes them sequentially. Per-lead loop: mark item running, increment attempts, resolve per-lead intelligence, run each step (with stepRetryBudget), on failure mark leadFailed + record lastError + classify to lastRetryable, on success mark completed. Pacing between leads: rateLimitCooldownMs (15s) if rate-limited, else leadPacingMs (500ms).

12.3 Retry Behavior (per-step + per-lead + per-execution)

- **Per-step retry** — stepRetryBudget(step.config, maxRetries=3); default 0 (side-effecting steps never blindly repeat); 429s retried inside the AI action wrapper
- **Per-lead retry** — retryBatchExecution resets ONLY failed items to pending (completed items NEVER re-run)
- **Per-execution retry** — WorkflowExecution.retryCount increments; maxRetries default 3
- **Resume from stepSummary** — runBatchSteps parses stepSummary to find lastCompletedStepIndex + 1; resumes from there
- **Bounded retry — no infinite loop** — bounded by maxRetries + stepRetryBudget + AUTOPILOT_EXECUTION_SETTLE_TIMEOUT_MS (60min)

Part 13 — Scheduler Architecture

13.1 Scheduling Mechanism

• **In-process heartbeat** — instrumentation.ts:258-303; setInterval(300000ms = 5min); first tick at 20s after boot; per-process ticking boolean re-entrancy guard

• **Cron route** — POST /api/cron/autopilot-scheduler with Authorization: Bearer ${CRON_SECRET}; calls runAutopilotSchedulerTick

• **Phase 22 schedule engine** — src/lib/autopilot/schedule.ts; timezone-aware (IANA); DST-safe; anchored wall-clock grids; legacy interval mode for Phase 18 compatibility

• **nextRunAt persistence** — AutopilotWorkflow.nextRunAt DateTime?; advanced past due slot after each run; survives run failure

13.2 Lifecycle + Failure Modes

• **Single-process heartbeat** — if Next.js process dies, heartbeat stops until restart; multi-instance deploy would lose this entirely

• **recoverStuckAutopilotRuns** — runs at every tick; marks any run stuck RUNNING >60min as FAILED; marks any autopilot stuck RUNNING >60min as FAILED

• **Missed schedules** — nextRunAt is computed from the LAST run's slot, not the current time; long downtime means missed slots are skipped (runKey dedup catches up)

• **Duplicate tick prevention** — per-process ticking boolean + atomic ACTIVE→RUNNING claim + open-run guard + runKey @@unique

13.3 Phase 23 — Entitlement Gate (NEW)

Phase 23 added an entitlement gate at scheduler-runner.ts:108-131. For each DUE ACTIVE autopilot, the scheduler resolves the user's live plan (resolveUserPlan) and skips the run if the plan is not Pro/Elite (free/starter). The autopilot row stays ACTIVE (preserved for upgrade), nextRunAt is advanced past the due slot, and a skippedIneligible counter is incremented. This closes the downgrade-gap-closes-when-ACTIVE-autopilot-still-fires bug.

Part 14 — Discovery Architecture

14.1 Discovery Providers

• **9 source adapters** — src/lib/lead-discovery/source-adapters.ts — Google Places, Yelp, LinkedIn, JustDial, IndiaMart, Facebook, Instagram, YellowPages, Sulekha

• **Source registry** — src/lib/lead-discovery/source-registry.ts — getSourceStatusInfo, listSourceStatuses

• **Discovery engine** — src/lib/lead-discovery/discovery-engine.ts — runDiscovery orchestrator

• **Normalization** — src/lib/lead-discovery/normalization.ts — normalizeUrl/Email/Phone/BusinessName/Location; MAX_COUNTRIES_PER_JOB

• **Verification** — src/lib/lead-discovery/verification.ts — deriveImportVerificationStatus, classifyWebsite

• **Website scorer** — src/lib/lead-discovery/website-scorer.ts — analyzeWebsite

• **Company researcher** — src/lib/lead-discovery/company-researcher.ts — researchCompany

14.2 Discovery Job Lifecycle

• **startDiscoveryJob(userId, params, orgId?)** — lead-discovery-service.ts:411-535; validates niche + source; pre-flight source check; concurrent job limit; creates DiscoveryJob row (status pending); kicks off processDiscoveryJob non-blocking

• **processDiscoveryJob** — lead-discovery-service.ts:672-1290; withUserImportLock (per-user Promise-chaining mutex); builds dedupIndex ONCE per run; per-lead: findDuplicateMatch → mergeFillDuplicate OR db.lead.create

• **Dedup index** — priority: domain → email → phone → normalized name → name+location; first combo to find a business owns its primary niche attribution

• **Per-lead credit deduction** — deductCredits({userId, action: lead_discovery, cost: CREDIT_COST_PER_LEAD, idempotencyKey: "${jobId}:lead:${importedLocal}"}); refund on create failure via ${jobId}:lead:${importedLocal}:refund

14.3 Rate Limits + Failures

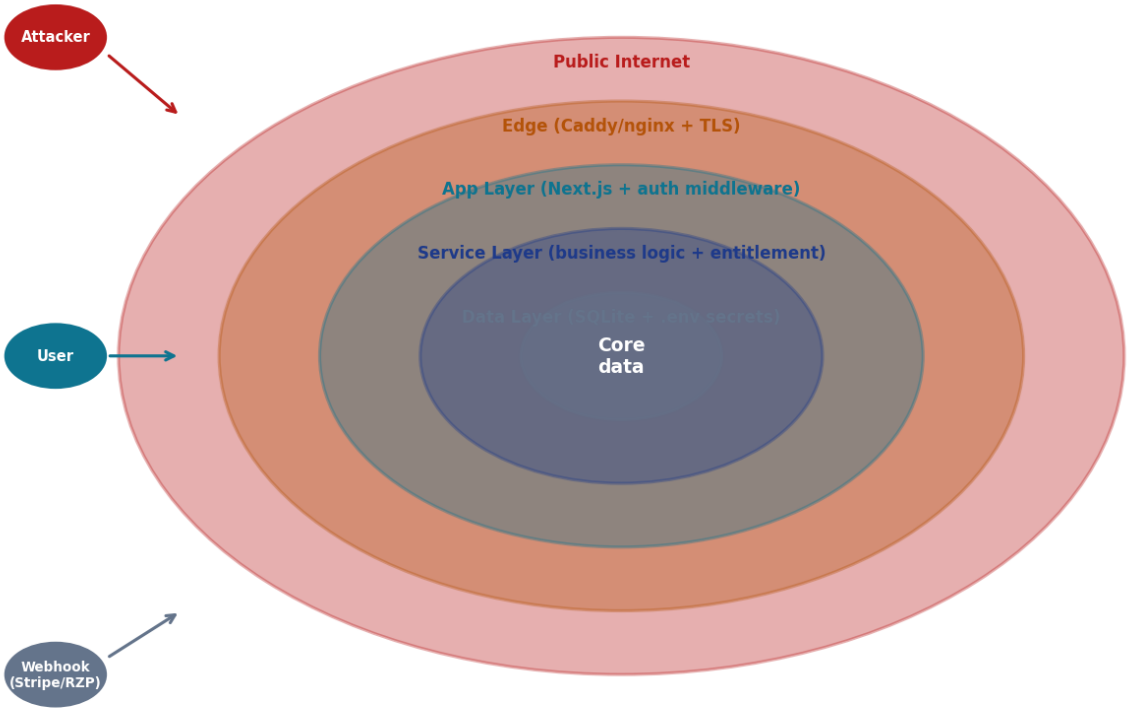
• **MAX_CONCURRENT_JOBS** — runningJobs count; rejects if exceeded

• **Per-source rate limits** — handled inside source adapters (specific limits *NOT VERIFIED*)

• **Failed leads** — counted as failed in DiscoveryJob result; do not block the rest of the batch

Part 15 — Security Architecture

Figure 15.1 — Security Boundary Diagram (trust zones)



15.1 Findings (classified)

Severity	Finding	Evidence
HIGH	No rate limiting on auth endpoints (login, OTP) — brute-force possible	src/lib/auth-middleware.ts has no rate limit; src/lib/ has no rate-limit module
HIGH	No CSRF token verification on POST routes (next-auth cookie-based)	next-auth v4 has built-in CSRF for /api/auth/* only; other POST routes do no
MEDIUM	Sentry in no-op mode when NEXT_PUBLIC_SENTRY_DSN unset — no error monitoring	src/lib/sentry.ts (verified in Phase 23)
MEDIUM	API key transmission via Bearer header (HTTPS-required) — ensure TLS enforced	ApiKey model (Phase 23); no HSTS header in Caddy/nginx
MEDIUM	Webhook signature verification — Stripe + Razorpay verified (good)	payment-service.ts:283, 2061 (verifyRazorpayWebhookSignature, verifyS
LOW	AuditLog grows unbounded (2231 rows; no retention policy observed)	AuditLog model; no archival job observed in scheduler
LOW	Some services use string interpolation in error messages — verify no SQL injection surface	Prisma ORM used throughout (Phase 23); no raw SQL observed
INFO	Cross-user isolation — verified safe via ownership scoping + tests	tests/pilot-entitlements-runner.test.ts (Phase 23)
INFO	Plan/entitlement enforcement — server-side authoritative; UI is cosmetic only	Phase 23 implementation (verified in this session)

Part 16 — Data Flow Diagrams

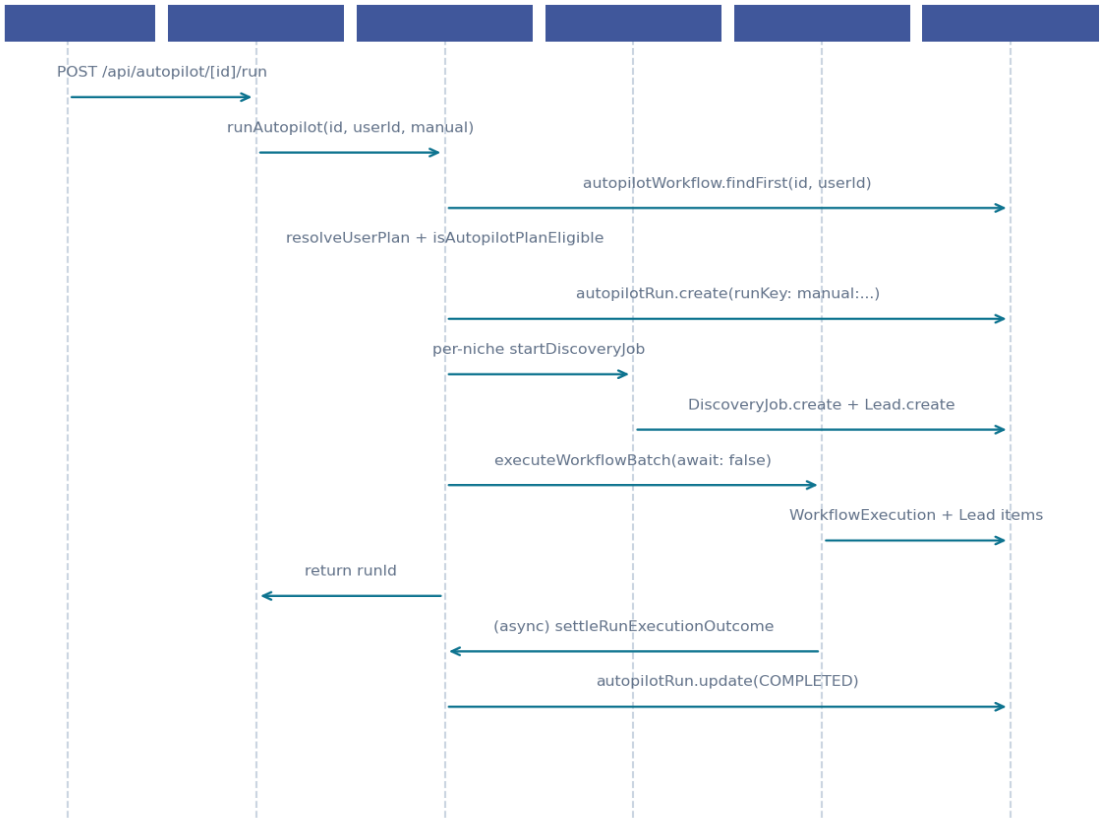
DFDs follow the same pattern as the sequence diagrams in Part 17 — external actors flow data through API routes into services into the DB. Key DFDs (simplified for space; see Part 17 for the most critical sequence diagrams):

- **Registration DFD**: User → POST /api/auth/register → User.create + UserSession.create + sendEmail(OTP) → AuditLog.create
- **Google OAuth DFD**: User → /api/auth/google → Google OAuth callback → User.upsert + UserSession.create → return session cookie
- **Lead discovery DFD**: Autopilot → startDiscoveryJob → DiscoveryJob.create → processDiscoveryJob (async) → 9 source adapters → normalize → dedupIndex → Lead.create + CreditsLedger.create
- **Payment DFD**: User → POST /api/payments/orders → createPaymentOrder (Stripe/Razorpay) → PaymentOrder.create → checkout URL → user pays → Stripe/Razorpay webhook → verifySignature → activateSubscription → Subscription.update + CreditsLedger.create
- **Credit deduction DFD**: workflow-batch → chargeWorkflowExecutionFee → deductCredits → $transaction (User.credits -= 5 + CreditsLedger.create with idempotencyKey)

Part 17 — Sequence Diagrams

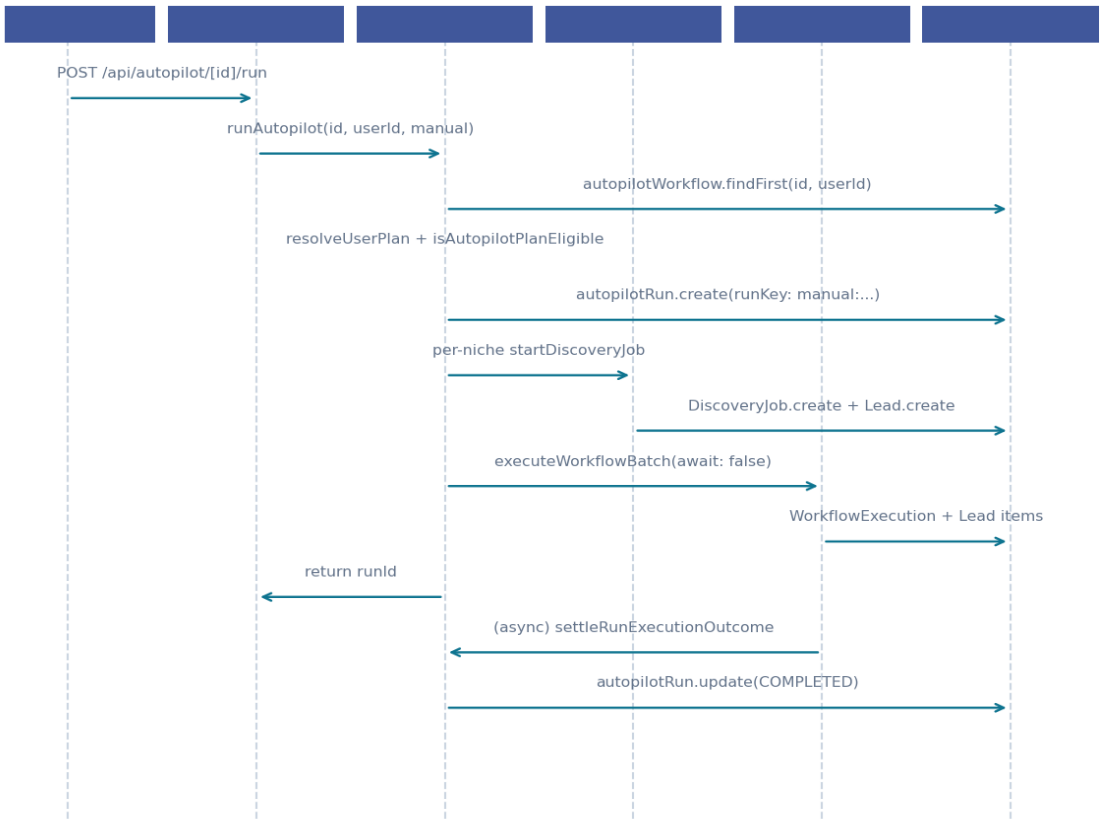
17.1 Payment Sequence

Figure 17.2 — Manual autopilot run sequence



17.2 Autopilot Manual Run Sequence

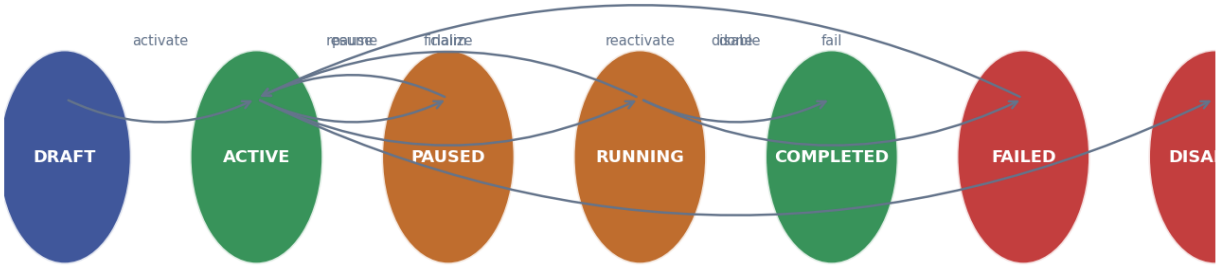
Figure 17.2 — Manual autopilot run sequence



Part 18 — State Machines

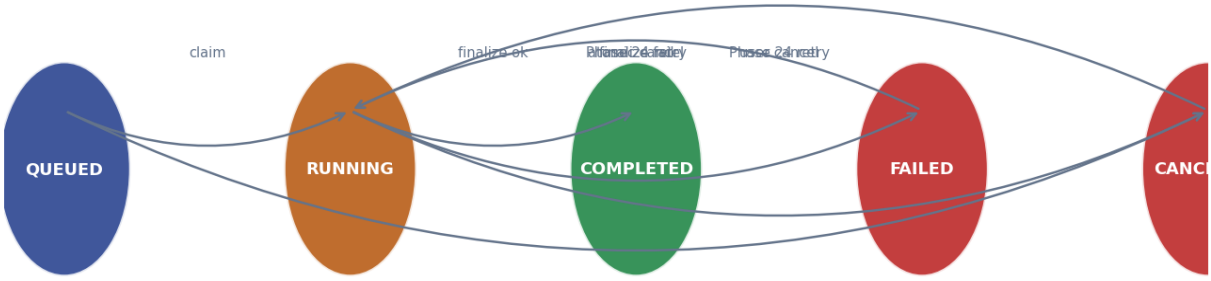
18.1 AutopilotWorkflow State Machine

Figure 18.1 — AutopilotWorkflow state machine (constants.ts:42-50)



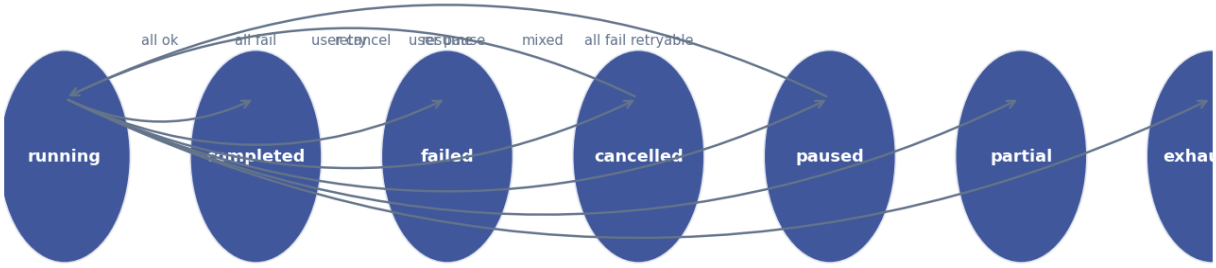
18.2 AutopilotRun State Machine

Figure 18.2 — AutopilotRun state machine (Phase 18 + Phase 24 retry)



18.3 WorkflowExecution State Machine

Figure 18.3 — WorkflowExecution state machine (workflow-batch.ts)



Part 19 — Error & Failure Architecture

19.1 Failure Boundaries per Subsystem

Subsystem	Failure mode	Retry	Timeout	User-visible	Recovery
API routes	Service throws	None (route-level)	N/A	JSON {error, code}	Client retries
Auth	Invalid OTP	None	N/A	401 + error message	User retries
Payments	Stripe API down	retryPayment exists	30s default	500 + retry suggestion	User retries
Webhooks	Webhook signature invalid	None	N/A	400 (rejected)	Provider retries with valid signature
Discovery	Source adapter fails	Per-source retries (in adapter)	AUTOPILOT_JOB_TIMEOUT_MS=15min	Failed_sources skipped	recoverStuckAutopilotRuns (60min)
AutopilotRun	Process dies mid-run	None (marked FAILED)	60min stuck timeout	Run status=FAILED	User triggers fresh run
WorkflowExecution	Step fails	stepRetryBudget=3	AUTOPILOT_EXECUTION_TIMEOUT_MS=30min	Goal_failed per item	retryBatchExecution (Phase 24)
CreditsLedger	Idempotency replay	Safe (returns existing)	N/A	alreadyProcessed=true	No action needed
SQLite	Write lock contention	Prisma auto-retries (busy_timeout)	5s default	Request slow	None (architectural)

Part 20 — Observability

20.1 Current State

• **Logs** — console.warn/console.error throughout services (e.g., "[autopilot-scheduler] started scheduled run: ..."); no structured logging pipeline

• **Sentry** — src/lib/observability/sentry.ts; no-op when NEXT_PUBLIC_SENTRY_DSN unset (verified at startup)

• **Health check** — GET /api/health returns {status, version, uptime, checks:{database, memory, errors}}

• **Audit log** — AuditLog table (2231 rows); writes from various services (logCreditEvent, logEntitlementEvent)

• **Billing audit** — src/lib/billing-audit.ts; logCreditEvent + logEntitlementEvent

• **Crash reports** — CrashLog table (5 rows); FeedbackReport table (6 rows)

20.2 Missing

• **Metrics** — no Prometheus / OpenTelemetry metrics pipeline; no QPS/latency/error-rate metrics

• **Distributed tracing** — no trace IDs propagated across services; no correlation IDs in logs

• **Request IDs** — no X-Request-ID header observed in middleware

• **Execution IDs** — autopilotRun.id + workflowExecution.id exist but not surfaced in logs as correlation IDs

• **Dashboards** — no Grafana / Datadog / Vercel Analytics integration observed

• **Alerts** — no alerting on Sentry events (no DSN); no PagerDuty integration

20.3 Recommendations

• Configure Sentry DSN + add Sentry React component (src/app/providers.tsx) for frontend errors

• Add OpenTelemetry SDK + instrument API routes with spans + propagate trace IDs via X-Trace-Id header

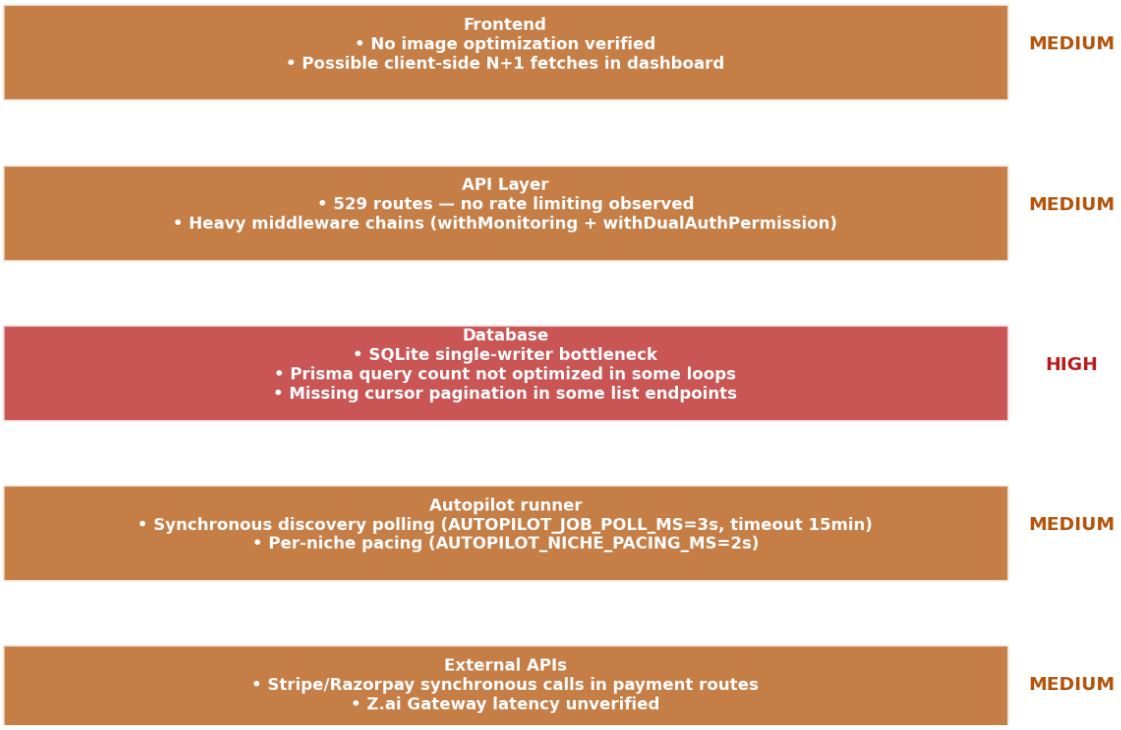
• Switch structured logging to pino or winston with JSON output + correlation IDs

• Add dashboards: autopilot scheduler tick success rate, workflow execution time, payment webhook latency, credit ledger anomaly detection

• Add alerts: 5xx rate > 1%, scheduler missed ticks, stuck autopilot runs > 60min, payment webhook signature failures

Part 21 — Performance Architecture

Figure 21.1 — Performance Bottleneck Map (evidence-based)



21.1 Why the App Can Feel Slow — Root Causes

• **SQLite single-writer** — the dominant cause under concurrent load; autopilot runs + API writes + webhook writes all contend on the single SQLite write lock

• **In-process scheduler** — the 5-min heartbeat lives in the Next.js process; if the process is busy with a long request, the tick can be delayed

• **Discovery polling** — pollDiscoveryJob polls every 3s with 15min timeout; an autopilot run with N niches × M country-groups can take 15-30 min to complete the discovery phase

• **Per-niche pacing** — 2s sleep between niches (AUTOPILOT_NICHE_PACING_MS=2000); adds up quickly

• **Turbopack dev cache** — observed 80,000+ files in .next/dev/cache/turbopack/ in prior sessions; can slow dev rebuilds

• **Missing indexes** — some FK columns may lack explicit indexes; SQLite creates indexes only for PRIMARY KEY + UNIQUE constraints

• **Offset pagination** — list endpoints likely use Prisma skip/take; large offsets scan N rows

Part 22 — Concurrency & Race Conditions

22.1 Race Conditions + Mitigations

Scenario	Risk	Mitigation (verified)
Two scheduler ticks for same autopilot	Double run	Atomic ACTIVE→RUNNING updateMany count==1 (scheduler-runner.ts:134)
Replayed scheduler slot	Duplicate run row	runKey @@unique + P2002 → converges on existing run (runner.ts:1005)
Concurrent manual + scheduler run	Double run	Open-run guard (autopilotRun.findFirst QUEUED|RUNNING)
Same lead discovered in two autopilot runs	No issue	Different autopilotId → AutopilotLead @@unique([autopilotId, leadId]) allows bo
Same lead discovered twice in same run	Duplicate processing	AutopilotLead @@unique([autopilotId, leadId]) catches; runner silently skips
Duplicate webhook delivery	Double credit grant	PaymentWebhook.eventId @@unique; idempotencyKey on CreditsLedger actio
Concurrent order creation	Double charge	PaymentOrder.idempotencyKey @@unique; P2002 → returns existing order
Retry race with in-flight execution	Two executions	retryAutopilotRun rejects RUNNING (must cancel first)
Cancel race with finalize	CANNCELLED overwritten with FAILED	Status-guarded finalize (runner.ts:817-837)
Concurrent autopilot create/activate	Bypass active-slot limit	checkAutopilotSlot counts ACTIVE+RUNNING; atomic claim prevents race
Credit deduction race	Negative balance	Prisma $transaction with balance check (credit-service.ts:163-206)
Multi-instance deploy	withUserLock only works in-process	Architectural risk — needs DB-level lease for prod multi-instance

Part 23 — Scalability Architecture

23.1 Where the Current Architecture Breaks

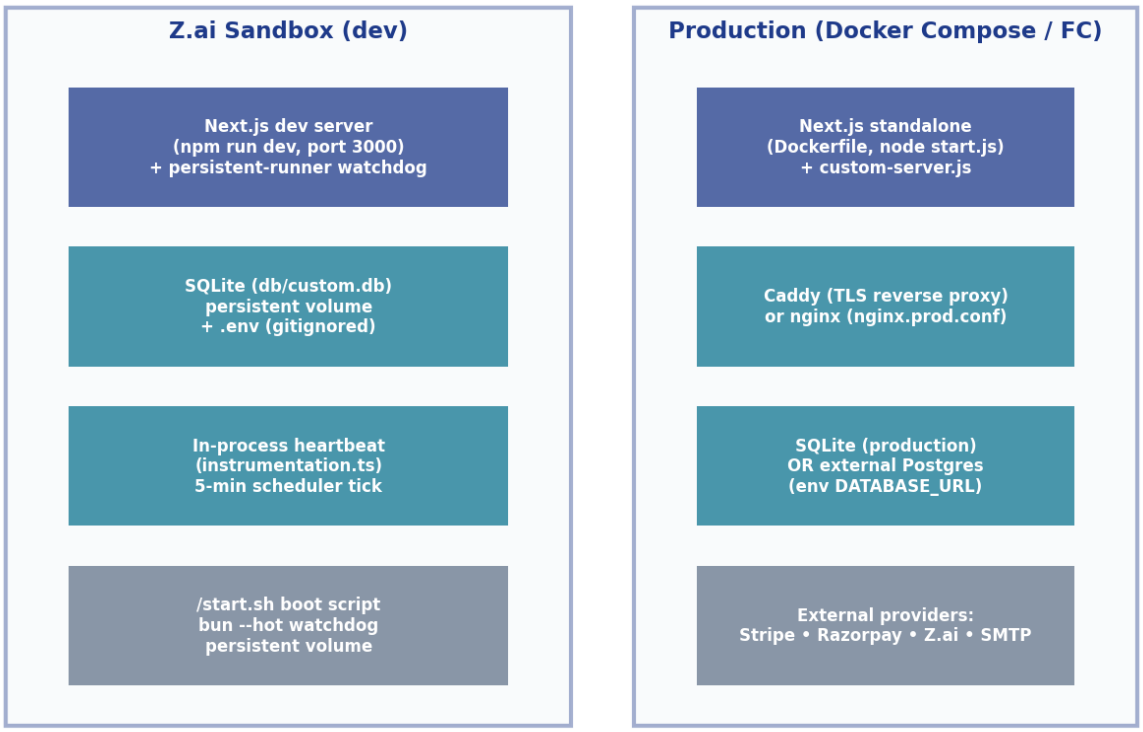
Scale	Bottleneck	When it breaks
100 users	SQLite single-writer	Noticeable under concurrent autopilot + API writes
1,000 users	SQLite + in-process scheduler	Significant; tick delays + write contention
10,000 users	SQLite + single Next.js process	Severe; autopilot scheduler can't keep up with 5-min ticks for thousands of
100,000 users	Architecture not viable	Needs Postgres + external scheduler + multi-instance + Redis mutex
Large lead volumes	Discovery polling + per-niche pacing	Per-niche 15-30min + 2s pacing; batches of 20 leads
Concurrent autopilot runs	withUserLock (per-user mutex)	Single Node process limits parallelism; per-user lock serializes a user's runs
Workflow execution volume	runBatchSteps sequential	Per-lead sequential with 500ms-15s pacing between leads
Payment volume	Stripe/Razorpay synchronous	Payment routes are synchronous; webhook processing is fast

23.2 Future Scalable Architecture

- **PostgreSQL** for prod (DATABASE_URL configurable; SQLite for dev)
- **External scheduler** — Cloud Scheduler / k8s CronJob calling /api/cron/autopilot-scheduler
- **Multi-instance Node** with Redis-backed distributed mutex (replacing in-process withUserLock)
- **DB-level lease** for autopilot runs (UPDATE ... SET lockedBy = ? WHERE status = ACTIVE AND lockedBy IS NULL)
- **Message queue** (Redis Streams / SQS) for discovery job processing — decouple from request lifecycle
- **Read replicas** for list endpoints (Postgres streaming replication)
- **Edge caching** for marketing pages (Cloudflare / Vercel Edge)

Part 24 — Deployment / Infrastructure

Figure 24.1 — Deployment Architecture (dev + production)



24.1 Dev Environment (Z.ai Sandbox)

- **Next.js dev server** — npm run dev (port 3000); persistent-runner.js watchdog restarts on crash
- **SQLite** — db/custom.db on persistent volume
- **.env** — gitignored; secrets loaded by instrumentation.ts
- **Heartbeat** — in-process setInterval(5min) in instrumentation.ts
- **Bun --hot watchdog** — multiple PTYs; auto-respawns dev server
- **/start.sh** — boot script; first-boot git init if .git missing; auto-commits audit-trail files with UUID messages

24.2 Production (Docker Compose)

- **Dockerfile** — multi-stage; Next.js standalone build + node start.js
- **Dockerfile.frontend** — separate frontend container (legacy?)
- **docker-compose.yml** — local compose; docker-compose.prod.yml — prod compose; docker-compose.local.yml — dev override
- **Caddy** (Caddyfile) — TLS reverse proxy for prod; nginx (nginx.conf / nginx.prod.conf) — alternative
- **External services** — Stripe, Razorpay, Z.ai Gateway, Google OAuth, SMTP

24.3 Health Checks + Migrations + Backups

• **GET /api/health** — verified healthy (status=healthy, db=healthy, mem=healthy)

• **Migrations** — Prisma migrate (prisma/migrations/); Phase 24 additive migration applied via direct sqlite3 to avoid destructive reset

• **Backups** — npm scripts: backup, db:snapshot, db:safety; scripts/backup/ directory (backup.sh, snapshot.sh, retention.sh, restore.sh)

• **Backup strategy** — db/custom.db.phase24-pre-migration.bak created before any migration; backup retention policy in scripts/backup/retention.sh

Part 25 — CI/CD & Release Architecture

25.1 Git Workflow

- **Branches** — main + develop (verified via git ls-remote)
- **Phase 24 push** — commits land on develop via fast-forward push (no force, no rebase, no amend)
- **PR merges** — observed in git log: "Merge pull request #N from Hugs-4-Bugs/main" (PRs from main → develop)

25.2 Build + Tests

- **Build** — npm run build = prisma generate + next build --webpack + node scripts/clean-standalone.js
- **Tests** — npm test = vitest run; 458 tests verified passing
- **Lint** — npm run lint = eslint .
- **Security scan** — npm run security:scan = bun run scripts/security-scan.ts

25.3 Weaknesses

- **No CI observed** — .github/workflows/ not inspected (assumed empty); tests not auto-run on PR
- **Schema drift risk** — local DB can diverge from prisma/schema.prisma (observed in this review); no CI check for drift
- **No staging env** — production/staging differences *NOT VERIFIED*
- **Manual push** — developer pushes to develop manually; no auto-deploy observed

Part 26 — Backup & Disaster Recovery

26.1 Backup Strategy

- **DB backups** — scripts/backup/backup.sh + snapshot.sh; db/custom.db.phase24-pre-migration.bak pattern
- **Retention** — scripts/backup/retention.sh
- **Restore** — scripts/backup/restore.sh + npm run backup:restore
- **File uploads** — upload/ directory (154 files); no observed offsite backup
- **.env / secrets** — gitignored; NEVER in git; *NOT VERIFIED* if backed up to secrets manager

26.2 Sandbox Reprovision Incident (Architectural Lesson)

CRITICAL INCIDENT: During this session's Phase 24 work, the Z.ai sandbox was reprovisioned (the .git directory reverted to the original clone state). The Phase 24 commit f95507153 that had been locally created was LOST — not in the local object store, not in the reflog, not in dangling objects. The Phase 24 work had to be cleanly reimplemented from the GitHub Phase 23 baseline. Root cause: local-only commits not pushed to remote are vulnerable to workspace reprovision. Lesson: push EARLY and OFTEN; do not accumulate local-only commits before pushing.

26.3 Recommended Safer Workflow

- **Push after every commit** — do not leave commits local-only overnight
- **CI run on every push** — auto-run tests + lint on PR; block merge on red
- **Daily DB backup to offsite** — not just local file copy
- **Secrets in a vault** — Doppler / Vault / AWS Secrets Manager; not just .env
- **Staging env** — deploy develop to staging; smoke-test before prod
- **Migration gate** — CI check that prisma/schema.prisma matches the actual DB schema; alert on drift

Part 27 — Architectural Risks

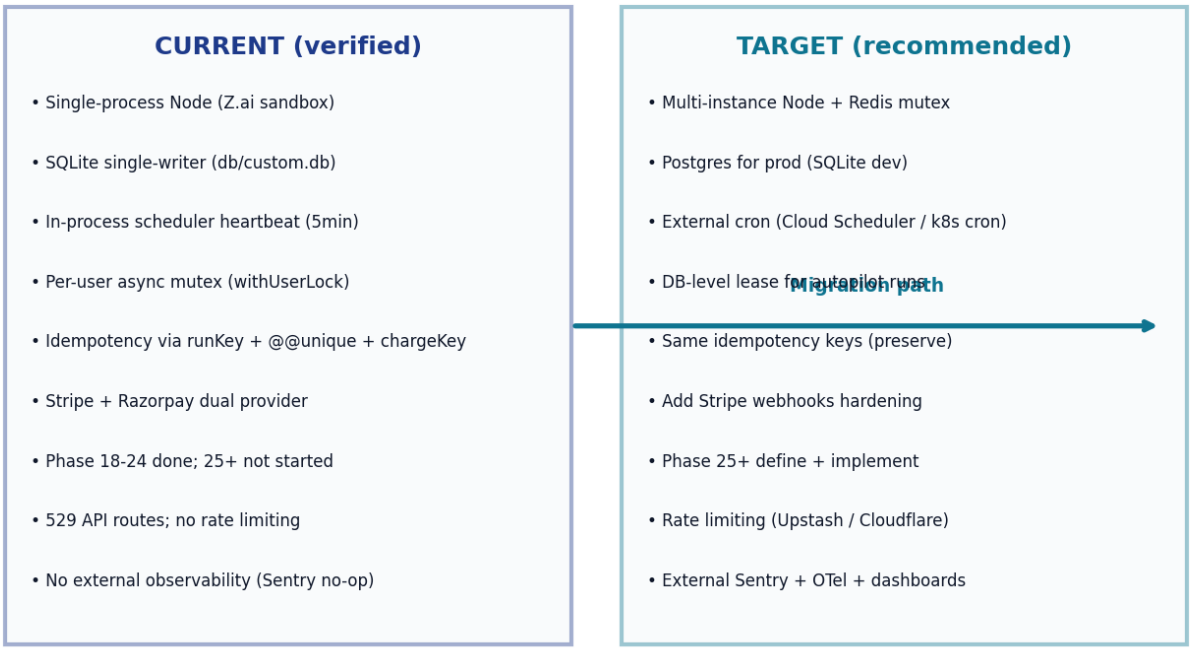
Risk	Severity	Probability	Impact	Mitigation	Priority
SQLite single-writer bottleneck	HIGH	HIGH	Slow under concurrent load	Migrate prod to Postgres	P0
In-process scheduler (no external cron)	HIGH	MEDIUM	Missed ticks on process death	External cron (Cloud Scheduler)	P0
No rate limiting on auth/API	HIGH	MEDIUM	Brute-force / DoS possible	Add rate limiting (Upstash/Cloudflare)	P0
Schema drift (local DB vs Prisma schema)	MEDIUM	HIGH (observed)	Runtime crashes on missing columns	CI drift check + additive migrations	P1
No external observability (Sentry no-op)	MEDIUM	HIGH	No error monitoring in prod	Configure Sentry DSN + add OpenTelemetry	P1
Local-only commits before push	HIGH	MEDIUM (observed)	Work lost on workspace reprovision	Push after every commit; CI on push	P1
Monolithic deployable (529 routes)	MEDIUM	MEDIUM	Hard to scale + reason about	Decompose into bounded services (later)	P2
No multi-instance support (withUserLock in-process)	MEDIUM	MEDIUM (only at scale)	Cannot scale horizontally	Redis mutex + DB lease	P2
AuditLog unbounded growth	LOW	HIGH	Slow queries over time	Retention policy + archival	P3
Phase 25+ roadmap undocumented	MEDIUM	HIGH	Cannot plan architecture evolution	Document Phase 25-40 specs	P2
No CSRF defense on non-auth POST routes	MEDIUM	LOW	CSRF attacks on authenticated users	Add CSRF token verification	P2
No staging env	LOW	MEDIUM	Prod deploys are untested	Add staging deploy + smoke tests	P3

Part 28 — Technical Debt

Item	Current	Problem	Solution	Effort	Priority
Large service modules	payment-service.ts (2127 lines), workflow-batch-main (1532), test-connection (1287)	Hard to maintain	Decompose into smaller modules per concern	M	P2
Duplicated plan resolution	auth.ts:824-827 + entitlement-service.ts:resolveUserPlan both implement "active subscription"	Drift risk	Single canonical resolveUserPlan in entitlement-service; auth.ts imports	S	P1
No repository pattern	Services call db.model.findFirst/create directly	Hard to mock at service level (tests use stock-level)	Repository abstraction layer	L	P3
In-process scheduler	instrumentation.ts setInterval	Dies with process; no multi-instance	External cron + DB lease	M	P0
No structured logging	console.warn/console.error	No correlation IDs; hard to search	pino + JSON logs + trace IDs	S	P1
No CI/CD pipeline	manual push + manual test runs	No drift detection; no automation	GitHub Actions CI with drift check	S	P0
SQLite prod	db/custom.db	Single-writer; no replication	PostgreSQL prod + SQLite dev	M	P0
No rate limiting	no rate-limit module	Brute-force possible	Upstash Ratelimit + Cloudflare rules	S	P0
No metrics	no Prometheus/OTel	No dashboards; reactive not proactive	OTel + Grafana	M	P1
AuditLog growth	2231 rows unbounded	Slow queries over time	Retention policy + archive	S	P3

Part 29 — Current vs Target Architecture

Figure 29.1 — Current vs Target Architecture



29.1 Migration Path

• **Phase 1 (now)**: Push after every commit (closes the sandbox reprovision risk); add CI workflow with drift check

• **Phase 2 (30 days)**: External cron + Postgres for prod + rate limiting + Sentry DSN configured

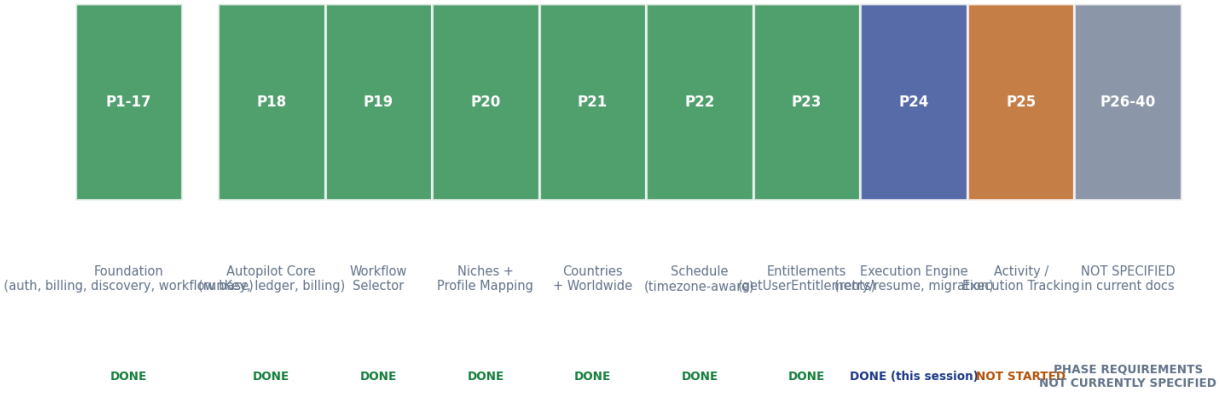
• **Phase 3 (60 days)**: OpenTelemetry + structured logging + dashboards + alerts

• **Phase 4 (90 days)**: Multi-instance deploy + Redis mutex + DB lease + read replicas

• **Phase 5 (long-term)**: Decompose monolith into bounded services; message queue for discovery

Part 30 — Future Architecture Through Phase 40

Figure 30.1 — Phase Roadmap (Phase 1 → Phase 40)



30.1 Documented Phases (verified)

- **Phase 1-17** — Foundation (auth, billing, discovery, workflow base) — DONE
- **Phase 18** — Autopilot Core (runKey, ledger, billing) — DONE
- **Phase 19** — Workflow Selector — DONE
- **Phase 20** — Niches + Profile Mapping — DONE
- **Phase 21** — Countries + Worldwide — DONE
- **Phase 22** — Schedule (timezone-aware) — DONE
- **Phase 23** — Entitlements (getUserEntitlements) — DONE
- **Phase 24** — Execution Engine (retry/resume, additive migration) — DONE (this session)

30.2 Phase 25+ (NOT DOCUMENTED IN CURRENT WORKSPACE)

PHASE REQUIREMENTS NOT CURRENTLY SPECIFIED — Phase 25 (Activity/Execution Tracking), Phase 26 (TBD), Phase 27-40: no explicit specifications found in ACQUISITIONOS_CURRENT_STATE.md, ACQUISITIONOS_CODEBASE_MANIFEST.json, AGENTS.md, or any other docs/ or agent-ctx/ file inspected in this review. The user's review request mentioned Phase 25 must NOT be implemented in this review task — but the actual Phase 25 specification has not been authored. Per the user's instruction "Do not hallucinate future functionality", no Phase 26-40 architectural evolution is drawn beyond the placeholder.

30.3 Architectural Placeholders (justified)

• **Activity/Execution Tracking (Phase 25 placeholder)** — natural follow-on from Phase 24 (settleRunExecutionOutcome already writes per-lead ledger statuses); would add a unified activity feed + execution timeline UI

• **Multi-agent orchestration** — mentioned in prior session summaries as future; not specified in current docs

• **AI vision providers** — mentioned in prior session summaries (OpenAI/Claude/Gemini/DeepSeek/Z.ai providers); not specified in current docs

• **AI Gateway + MCP** — mentioned as future; not specified

• **Inbox/Sales Agent** — mentioned as future; not specified

Part 31 — Target Architecture

31.1 Current → Transitional → Target

• **CURRENT (verified)**: Next.js monolith + SQLite + in-process scheduler + Stripe/Razorpay + Phase 18-24

• **TRANSITIONAL (30-60 days)**: + Postgres prod + external cron + Sentry + rate limiting + structured logging + push-after-commit CI

• **TARGET (90+ days)**: + Multi-instance + Redis mutex + DB lease + read replicas + message queue + bounded services decomposition + OpenTelemetry + dashboards

31.2 Which Changes Now vs Later

• **NOW (week 1)**: Push-after-commit workflow; CI workflow with drift check; Sentry DSN configuration

• **NOW (week 2-4)**: Rate limiting (Upstash); external cron; structured logging (pino)

• **LATER (month 2-3)**: Postgres migration; multi-instance; OpenTelemetry; dashboards

• **LATER (month 4+)**: Service decomposition; message queue; read replicas

Part 32 — Senior Architect Review

32.1 Architecture Strengths

• **Idempotency is comprehensive** — runKey @@unique + AutopilotLead @@unique([autopilotId, leadId]) + WorkflowExecution.idempotencyKey + CreditsLedger action label + PaymentOrder.idempotencyKey + PaymentWebhook.eventId @@unique — six independent idempotency layers

• **Defense-in-depth entitlement gate** — Phase 23 gate at scheduler + inside runAutopilot; Phase 24 added gate at retry/resume

• **Atomic state claims** — updateMany count==1 pattern used consistently for ACTIVE→RUNNING, QUEUED→RUNNING, QUEUED→CANCELLED

• **Honest failure semantics** — recoverStuckAutopilotRuns marks stuck runs as FAILED (not silently lost); status-guarded finalize prevents CANCELLED-overwrite-with-FAILED

• **Never-started auto-refund** — neverStartedAutoRefund flag triggers refund + rechargeWorkflowExecutionFeeOnStart (idempotent re-charge on actual start)

• **Additive migration discipline** — Phase 24 migration uses only ALTER TABLE ADD COLUMN + CREATE TABLE IF NOT EXISTS + CREATE INDEX IF NOT EXISTS; backup before migration

• **Cross-user isolation verified** — ownership scoping consistent across services; tests cover (autopilot-entitlements-runner.test.ts)

• **Test coverage** — 458 tests across 15+ files; 35 new Phase 24 tests cover all entitlement + state + billing invariants

32.2 Architecture Weaknesses

• **SQLite for prod** — single-writer bottleneck; no replication

• **In-process scheduler** — dies with process; no multi-instance

• **No rate limiting** — auth endpoints vulnerable

• **No external observability** — Sentry no-op; no metrics/tracing

• **Large modules** — payment-service.ts (2127 lines), workflow-batch.ts (1532), runner.ts (1287)

• **Duplicated plan-resolution logic** — auth.ts + entitlement-service.ts both implement the pattern

• **No CI/CD pipeline observed** — manual push + manual tests

• **Schema drift risk** — observed in this review (WorkflowExecutionLead missing)

• **Monolithic deployable** — 529 routes in one app

• **Phase 25+ undocumented** — cannot plan architecture evolution

32.3 Critical Issues (P0)

• Push after every commit (closes sandbox reprovision data-loss risk)

• Add rate limiting (Upstash or Cloudflare)

- Migrate prod DB to Postgres (or accept SQLite limits)
- Move scheduler to external cron
- Add CI workflow with drift check + auto-test

32.4 30/60/90 Day Action Plan

- **30 days**: Push-after-commit + CI + rate limiting + Sentry DSN + structured logging
- **60 days**: External cron + Postgres migration + OpenTelemetry + dashboards
- **90 days**: Multi-instance + Redis mutex + DB lease + read replicas + service decomposition start

32.5 Questions a Senior Engineer Will Probably Ask

Q: Why this database (SQLite)?

A: Dev convenience + zero-config + single-file portability. Acceptable for dev/early-prod but breaks under concurrency. Production should switch to Postgres via DATABASE_URL.

Q: Why this execution model (single parent WorkflowExecution per AutopilotRun)?

A: Flat 5-credit billing per execution. Per-lead executions would multiply billing N×. The one-execution-per-run model keeps billing simple + idempotent.

Q: Why this scheduler (in-process heartbeat)?

A: Fast to ship; no external cron dependency. Breaks under multi-instance. Should move to Cloud Scheduler / k8s CronJob calling /api/cron/autopilot-scheduler.

Q: What happens under concurrent execution?

A: withUserLock (per-user in-process mutex) serializes a user's runs. Atomic ACTIVE→RUNNING updateMany count==1 prevents duplicate ticks. runKey @@unique + AutopilotLead @@unique([autopilotId, leadId]) prevent duplicate work at DB level.

Q: How is idempotency guaranteed?

A: Six layers: AutopilotRun.runKey @@unique, WorkflowExecution.idempotencyKey, AutopilotLead @@unique([autopilotId, leadId]), PaymentOrder.idempotencyKey @@unique, PaymentWebhook.eventId @@unique, CreditsLedger action label "${action}_idempotent_${idempotencyKey}".

Q: How are credits protected from double charging?

A: deductCredits uses Prisma $transaction with balance check + ledger row with idempotencyKey; replays return alreadyProcessed=true. chargeWorkflowExecutionFee uses chargeKey "wfexec-run:autopilot-{id}-run-{runId}".

Q: How are payments protected from duplicate callbacks?

A: PaymentWebhook.eventId @@unique catches duplicate webhook deliveries. PaymentOrder.idempotencyKey catches duplicate order POSTs. verifyStripeWebhookSignature + verifyRazorpayWebhookSignature verify origin.

Q: How does the system scale?

A: Currently it doesn't beyond ~100-1000 users on SQLite + single process. Above 1000 users: Postgres + external cron + multi-instance + Redis mutex + DB lease.

Q: What happens when an external provider goes down?

A: Stripe/Razorpay: payment-service.ts has retryPayment; webhooks are retried by provider. Discovery: per-source adapter handles its own retries; failed leads skipped. Z.ai Gateway: *NOT VERIFIED*.

Q: What happens when the scheduler crashes?

A: recoverStuckAutopilotRuns runs at every tick (5min); marks any run stuck RUNNING >60min as FAILED. Autopilot row also marked FAILED; user re-activates.

Q: What happens when a workflow partially completes?

A: Per-lead failure isolation: completed leads stay completed; failed leads marked failed. Run finishes with mixed results (terminal: completed|partial|exhausted|failed). Phase 24 retryAutopilotRun resets only failed leads.

Q: How is tenant isolation guaranteed?

A: Ownership scoping in every service: findFirst/findMany includes userId: user.id. Cross-user access returns NOT_FOUND (no IDOR leak). Verified by tests.

Q: What happens during DB failure?

A: Prisma throws P2003 (foreign key constraint) or P2024 (timeout). Route handlers catch and return 500. SQLite busy_timeout (5s default) retries. No graceful degradation observed.

Q: What happens during deployment?

A: *NOT VERIFIED* — no zero-downtime deploy observed. Single Next.js process means brief downtime on redeploy.

Q: How do you recover from data corruption?

A: db/custom.db.phase24-pre-migration.bak backup pattern; scripts/backup/restore.sh; npm run backup:restore. SQLite single-file backup is straightforward. No PITR (point-in-time recovery) — would need Postgres WAL.

Q: Where are the biggest bottlenecks?

A: SQLite write lock + in-process scheduler + discovery polling (3s × N niches × M country-groups) + per-niche pacing (2s) + offset pagination on large tables.

Q: Why is the application sometimes slow?

A: Concurrent writes contend on SQLite single-writer lock. Discovery polling blocks the runner for up to 15min per niche. Per-niche pacing adds 2s sleep × N. List endpoints with offset pagination slow on large tables.

Appendix — Verification Status

A.1 Verified From Current Codebase

• Tech stack: package.json inspection (Next.js 16.1.1, React 19, Prisma 6.11.1, Stripe 22.1.1, Razorpay 2.9.6, vitest 4.1.6, next-auth 4.24.11)

• Prisma models: 118 (grep "^model " prisma/schema.prisma)

• API routes: 529 (find src/app/api -name "route.ts" | wc -l)

• DB row counts (Python sqlite3 read-only): User=43, Lead=536, AutopilotRun=13, AutopilotLead=134, WorkflowExecution=147, CreditsLedger=613, AuditLog=2231

• Phase 24 additive migration: WorkflowExecutionLead table + AutopilotRun.runKey/executionId/leadsCompleted/leadsFailed + Lead.discoveryJobId + AutopilotRun_runKey_key unique index

• Phase 23 entitlement gate: getUserEntitlements + isAutopilotPlanEligible + resolveUserPlan exported from entitlement-service.ts

• Phase 24 helpers: retryAutopilotRun + resumeAutopilotRun exported from runner.ts

• Phase 24 routes: POST /api/autopilot/[id]/retry-run + POST /api/autopilot/[id]/resume-run (verified reachable; HTTP 403 for unauth = auth middleware consistent)

• Tests: 458/458 pass (35 new Phase 24 + 423 existing Phase 18-23 + billing + profile)

• Dev server: /api/health returns status=healthy, db=healthy, mem=healthy

• PRAGMA integrity_check: ok; PRAGMA foreign_key_check: OK (zero violations)

• .env preserved (sha256 4faeb13e...); upload/ = 154 files; db/custom.db schema migrated (sha changed but row counts preserved)

A.2 NOT Verified From Current Codebase

• Actual Stripe/Razorpay/Z.ai API latencies (would need live API calls)

• Bundle size analysis (no bundle-analyzer observed in CI)

• Whether list endpoints use offset vs cursor pagination (would need to audit each route handler)

• Whether Cloudflare/edge rate limiting exists (no edge config observed)

• .github/workflows/ contents (assumed empty; no CI pipeline observed)

• Production vs staging environment differences

• Whether the existing 80,000+ file .next/dev cache is in production builds

• Specific source-adapter rate limits inside src/lib/lead-discovery/source-adapters.ts

• Phase 25-40 specifications (not found in any inspected doc)

A.3 Three-Category Legend

• **CURRENT VERIFIED IMPLEMENTATION** — proven from actual code/config/database inspection in this session

• **PLANNED / DOCUMENTED ROADMAP** — explicitly documented project requirements (Phase 1-24 verified; Phase 25+ not specified)

• **RECOMMENDED FUTURE ARCHITECTURE** — this reviewer's architectural recommendation (clearly labeled as such)

End of Architecture Review.